

Software Architecture and Design Specification

Project: Task Manager Application

Version: 1.0

Authors: Arya M G, Aryan Burman, Arushi Punhani, Arnav Sharma (Team 6)

Date: 02-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Arya M G, Aryan Burman, Arushi Punhani, Arnav Sharma	Initial draft of the architecture and design for Task Manager Application v1.0

Approvals

Role	Name	Signature/Date
QA Lead	Arushi Punhani	Arushi Punhani / 02-10-2026
Dev Lead / Team Lead	Aryan Burman	Aryan Burman / 02-10-2026
Product Owner	Arya M G	Arya M G / 02-10-2026
Course Coordinator	Asha Kumari	Asha Kumari / 02-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Task Manager Application v1.0, a web/mobile application that lets individuals and teams create, organise, assign, track and report on tasks. It translates the requirements in the Task Manager Application SRS v1.0 (02-09-2026) into components, interfaces, data stores and key interaction flows, and is the design basis for the Software Test Plan (STP v1.0, 17-09-2026).

1.2 Scope

Covers user registration and authentication, task creation and lifecycle management, categorisation/tagging and search, task assignment and collaboration (comments, list sharing), notifications and reminders, reporting/dashboard and export, and administration with audit logging. Third-party calendar/email provider integration and payment/billing are out of scope for this version, consistent with the STP.

1.3 Audience

Developers, QA engineers, security auditors, the course coordinator and instructors, and future maintenance teams.

1.4 Definitions

UI (User Interface), API (Application Programming Interface), REST (Representational State Transfer), CRUD (Create, Read, Update, Delete), JWT (JSON Web Token), RBAC (Role-Based Access Control), TLS (Transport Layer Security), ADR (Architecture Decision Record), NFR (Non-Functional Requirement), SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), WCAG (Web Content Accessibility Guidelines), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege).

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including a UML component diagram, UML sequence diagrams, the architectural decision and its rationale, a STRIDE threat model, API design and requirement traceability. Section 3 describes the system structure; Section 4 describes detailed design (flows, APIs, error handling, UX).

2.2 Related Documents

Task Manager Application SRS v1.0 (02-09-2026, Team 6); Software Test Plan v1.0 (17-09-2026); Requirements Traceability Matrix (STP Section 13); WCAG 2.1 AA Guidelines.

3. Architecture

3.1 Goals & Constraints

Goals: secure (TM-SR-001 to 005), responsive (95th percentile response ≤ 2 s at 500 concurrent users), highly available (99.5% monthly uptime), scalable to 10,000 users / 1,000,000 tasks, accessible (WCAG 2.1 AA) and usable on desktop and mobile (≥ 360 px width).

Constraints: the system is limited to the features listed in the SRS; no third-party calendar/email integration and no payment functionality in v1.0; notifications are delivered through an external Notification/Email Service; deleted tasks must remain recoverable for 30 days; the project runs on a student-team schedule (test execution 25-Sep to 08-Oct-2026, UAT 10 to 13-Oct-2026).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
End users (task owners)	Ease of use, reliable reminders, data safety, fast search and filtering
Teammates / collaborators	Clear assignment, comments, correct view/edit permissions
Administrators	Account management, audit trail of admin actions
Developers	Modularity, clear module boundaries, testability
QA team	Testable interfaces, traceability to requirements, mockable external service
Course coordinator / instructors	Completeness, adherence to SRS and process, quality of documentation
Security auditors	Authentication, authorisation, transport security, logging

3.3 Component (UML) Diagram

Figure 1 shows the main components and their dependencies. The six business modules match the modules defined in the STP (AuthModule, TaskModule, CollabModule, NotifModule, ReportModule, AdminModule).

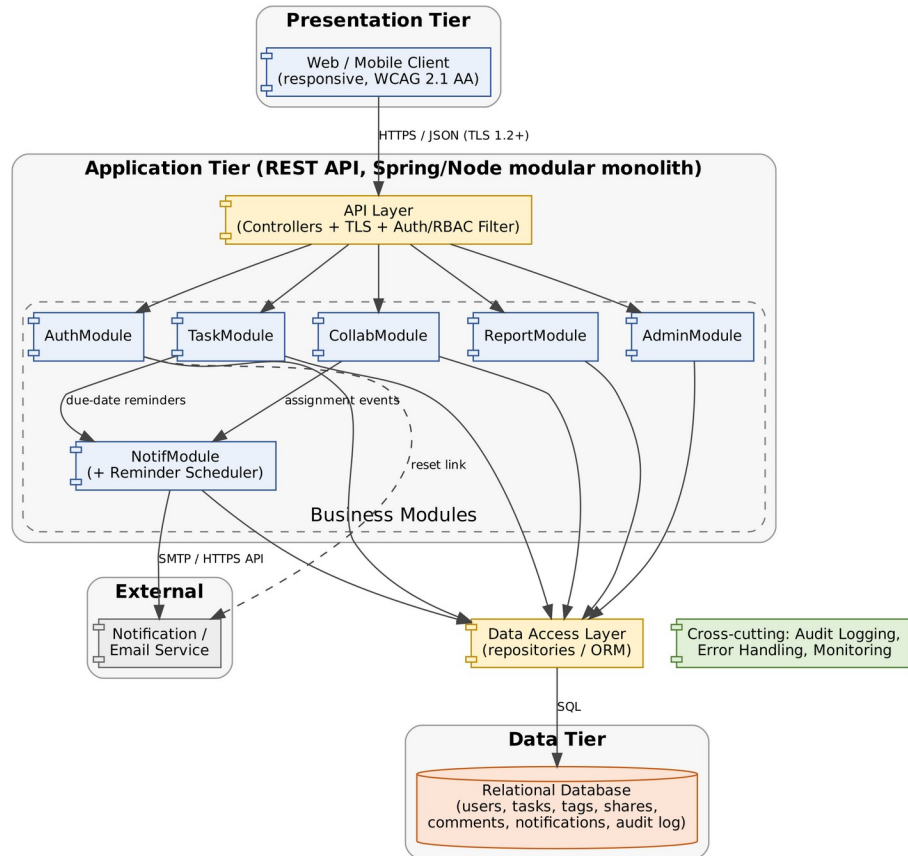


Figure 1: UML component diagram of the Task Manager Application

3.4 Component Descriptions

- **Web / Mobile Client:** responsive UI (≥ 360 px) for registration, login, task lists, search/filter/sort, collaboration, dashboard and admin screens; communicates only with the REST API over HTTPS.
- **API Layer:** REST controllers that terminate TLS 1.2+, validate the JWT, enforce RBAC and input validation, and route requests to the modules.
- **AuthModule:** registration, login, logout after 5 consecutive failed attempts, password hashing (bcrypt/Argon2), time-limited password-reset links, JWT issuance/expiry/invalidation (TM-F-001 to 004).
- **TaskModule:** task CRUD, complete/reopen, categories and tags, search, filter and sort; soft delete with 30-day recovery (TM-F-010 to 016).
- **CollabModule:** assignment to another registered user, comments on shared/assigned tasks, sharing a task list with view or edit permission (TM-F-020 to 022).
- **NotifModule:** in-app and email notifications; includes a Reminder Scheduler that sends reminders before due dates and notifies a teammate on assignment (TM-F-030, 031).
- **ReportModule:** dashboard counts by status and CSV/PDF export of a task list (TM-F-040, 041).
- **AdminModule:** view, disable or delete user accounts and read the audit log (TM-F-050, 051).
- **Cross-cutting services:** audit logging, standardised error handling, structured logging and monitoring used by all modules.
- **Data Access Layer:** repositories/ORM; the only component that talks to the database, so queries are parameterised and access is centralised.
- **Relational Database:** persistent storage for users, tasks, tags, lists and shares, comments, notifications and the audit log.
- **External Notification/Email Service:** third-party service for email delivery; a sandbox/mock is used during testing (see STP).

3.5 Chosen Architecture Pattern and Rationale

Layered architecture (presentation → API → business modules → data access → database), implemented as a modular monolith with one module per feature area. Layers and modules keep concerns separate, let each module be unit-tested on its own (as the STP plans), and keep deployment simple for the team.

Rejected alternative: microservices. Six modules sharing one relational schema and one team do not justify the operational complexity (service discovery, distributed transactions, per-service deployment). The API instances are stateless (JWT), so the monolith can still be scaled horizontally behind a load balancer to meet TM-NF-003. Module boundaries (each module exposes a service interface and does not read another module's tables directly) keep a later split into services possible.

3.6 Technology Stack & Data Stores

Note: the SRS/STP do not fix a technology stack; the following is the proposed stack for v1.0 and can be changed by the team without affecting the architecture.

Concern	Choice
Client	React single-page web app, responsive CSS (works on mobile browsers at ≥ 360 px); accessibility checked with axe/WAVE
Backend	Node.js with Express (REST/JSON), modular structure with one folder per module
Database	PostgreSQL (relational); indexes on owner, status, due date, priority and tag for search/filter/sort
Security	TLS 1.2+ (HTTPS only), bcrypt/Argon2 salted hashing, JWT with expiry, RBAC middleware
Notifications	External email service via API/SMTP (sandbox in test); scheduled job for reminders
Export	CSV generation and PDF generation library in ReportModule
Tooling (from STP)	Selenium, Postman, JMeter, Jira, axe/WAVE; PlantUML/draw.io for diagrams

Main data entities

Entity	Key attributes	Notes
User	id, email (unique), passwordHash, role (USER/ADMIN), status, failedCount, lockedUntil	Never stores plaintext passwords
Task	id, ownerId, assigneeId, title, description, dueDate, priority, status, createdAt, deletedAt	deletedAt supports 30-day recovery; purge job removes older rows
Category / Tag	id, ownerId, name; TaskTag(taskId, tagId)	Many-to-many tags per task
TaskList / ListShare	listId, ownerId, name; shareId, listId, userId, permission (VIEW/EDIT)	Drives RBAC for shared lists
Comment	id, taskId, authorId, text, createdAt	Only on shared or assigned tasks
Notification	id, userId, type, taskId, channel, sentAt, readAt	Reminder and assignment notifications
PasswordResetToken	id, userId, tokenHash, expiresAt	Time-limited, single use
AuditLog	id, actorId, action, targetId, timestamp, outcome	Admin actions and failed-login events

3.7 Risks & Mitigations

Risk	Mitigation
Slow queries as data grows (up to 1,000,000 tasks)	Indexes, server-side pagination, cached dashboard counts, JMeter load tests on scaled datasets
Downtime exceeds 99.5% monthly target	Stateless API instances behind a load balancer, health checks and auto-restart, regular database backups
External Notification/Email Service unavailable	Queue and retry failed sends; in-app notification is stored regardless; mock service for tests
Brute-force or credential-stuffing attacks	Lockout after 5 consecutive failures, audit logging and alerting, hashed passwords
Unauthorised access to other users' tasks	RBAC checks in the API Layer on every task and list request; 403 responses covered by security tests
Accidental data loss	Soft delete with 30-day retention, confirmation prompt before delete, backups
Delay in stable build for testing	Early smoke builds and a modular design that allows module-by-module delivery

3.8 Traceability to Requirements

Each requirement from the SRS/STP is mapped to the component that realises it and the design reference used in the RTM.

Req ID	Requirement	Component(s)	Design Ref
TM-F-001	User registration	AuthModule	DS-Auth-01
TM-F-002	Login with email and password	AuthModule	DS-Auth-02
TM-F-003	Lockout after 5 failed logins	AuthModule	DS-Auth-03
TM-F-004	Password reset via time-limited link	AuthModule, NotifModule	DS-Auth-04
TM-F-010	Create task	TaskModule	DS-Task-01
TM-F-011	Edit task	TaskModule	DS-Task-02
TM-F-012	Delete task with confirmation	TaskModule	DS-Task-03
TM-F-013	Complete / reopen task	TaskModule	DS-Task-04
TM-F-014	Categories and tags	TaskModule	DS-Task-05
TM-F-015	Search and filter	TaskModule, Data Access Layer	DS-Task-06
TM-F-016	Sort task list	TaskModule	DS-Task-07
TM-F-020	Assign task	CollabModule	DS-Collab-01
TM-F-021	Comments on shared task	CollabModule	DS-Collab-02
TM-F-022	Share list with permission	CollabModule	DS-Collab-03
TM-F-030	Due-date reminder	NotifModule (Scheduler)	DS-Notif-01
TM-F-031	Assignment notification	NotifModule, CollabModule	DS-Notif-02
TM-F-040	Dashboard summary	ReportModule	DS-Report-01
TM-F-041	Export CSV/PDF	ReportModule	DS-Report-02
TM-F-050	Admin user management	AdminModule	DS-Admin-01
TM-F-051	Audit logging of admin actions	AdminModule, Audit service	DS-Admin-02
TM-NF-001	Response time ≤ 2 s (p95, 500 users)	API Layer, indexes, pagination, caching	DS-Perf-01
TM-NF-002	99.5% monthly uptime	Deployment: load balancer, health checks, backups	DS-Ops-01
TM-NF-003	10,000 users / 1,000,000 tasks	Stateless API scaling, DB indexing	DS-Scale-01
TM-NF-004	WCAG 2.1 AA	Web client (Section 4.5)	DS-UX-01
TM-NF-005	30-day recoverable deletion	TaskModule (soft delete), purge job	DS-Data-01
TM-NF-006	Responsive UI (≥ 360 px)	Web client (Section 4.5)	DS-UX-02
TM-SR-001	TLS 1.2+	API Layer / infrastructure	DS-Sec-01
TM-SR-002	Salted password hashing	AuthModule	DS-Sec-02
TM-SR-003	RBAC	API Layer, TaskModule, CollabModule	DS-Sec-03
TM-SR-004	JWT expiry and invalidation on logout	AuthModule	DS-Sec-04
TM-SR-005	Failed-login logging and alerting	AuthModule, Audit service	DS-Sec-05

3.9 Security Architecture

Threat modelling (STRIDE):

Threat	Example in this system	Mitigation
Spoofing	Attacker logs in as another user or reuses a stolen token	Salted bcrypt/Argon2 hashes, logout after 5 failures, signed JWT with short expiry, invalidated on logout
Tampering	Modifying task data in transit or a JWT payload	TLS 1.2+ on all traffic, JWT signature verification, parameterised queries, server-side input validation
Repudiation	Admin denies deleting an account	Audit log of all admin actions with actor, target and timestamp
Information disclosure	Reading another user's tasks; password leakage	RBAC on every request (403 on unauthorised access), no plaintext passwords, no sensitive data in logs or error messages
Denial of service	Request floods or repeated login attempts	Rate limiting, logout, pagination limits, load balancer and health checks
Elevation of privilege	A normal user calls admin endpoints or edits a view-only list	Role checks (USER / ADMIN) and per-list VIEW/EDIT permission checks in the API Layer

Additional controls: password-reset tokens are time-limited and single use; JWT secrets and database credentials are kept in environment configuration, not in source code. These controls are verified by the security tests in STP Section 5.1.

4. Design

4.1 Design Overview

The application follows the layered design from Section 3: each request passes the API Layer (TLS, JWT validation, RBAC, input validation), is handled by one business module, and reaches the database only through the Data Access Layer. Modules communicate through service interfaces and events (for example CollabModule raises a TaskAssigned event consumed by NotifModule), which keeps them independently testable.

4.2 UML Sequence Diagrams

Two flows specific to this project are shown: user login with account lockout (Figure 2) and task assignment with notification (Figure 3).

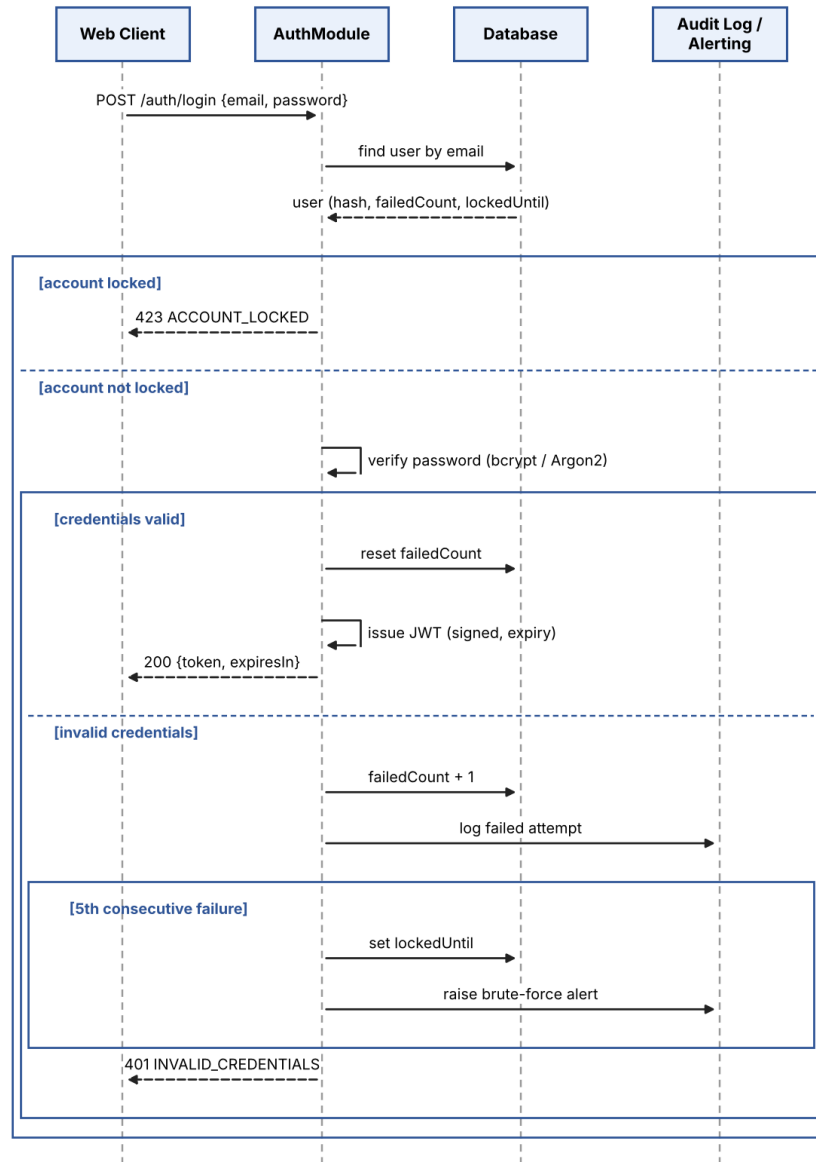


Figure 2: Sequence diagram, login with lockout and JWT issuance (TM-F-002, TM-F-003, TM-SR-004, TM-SR-005)

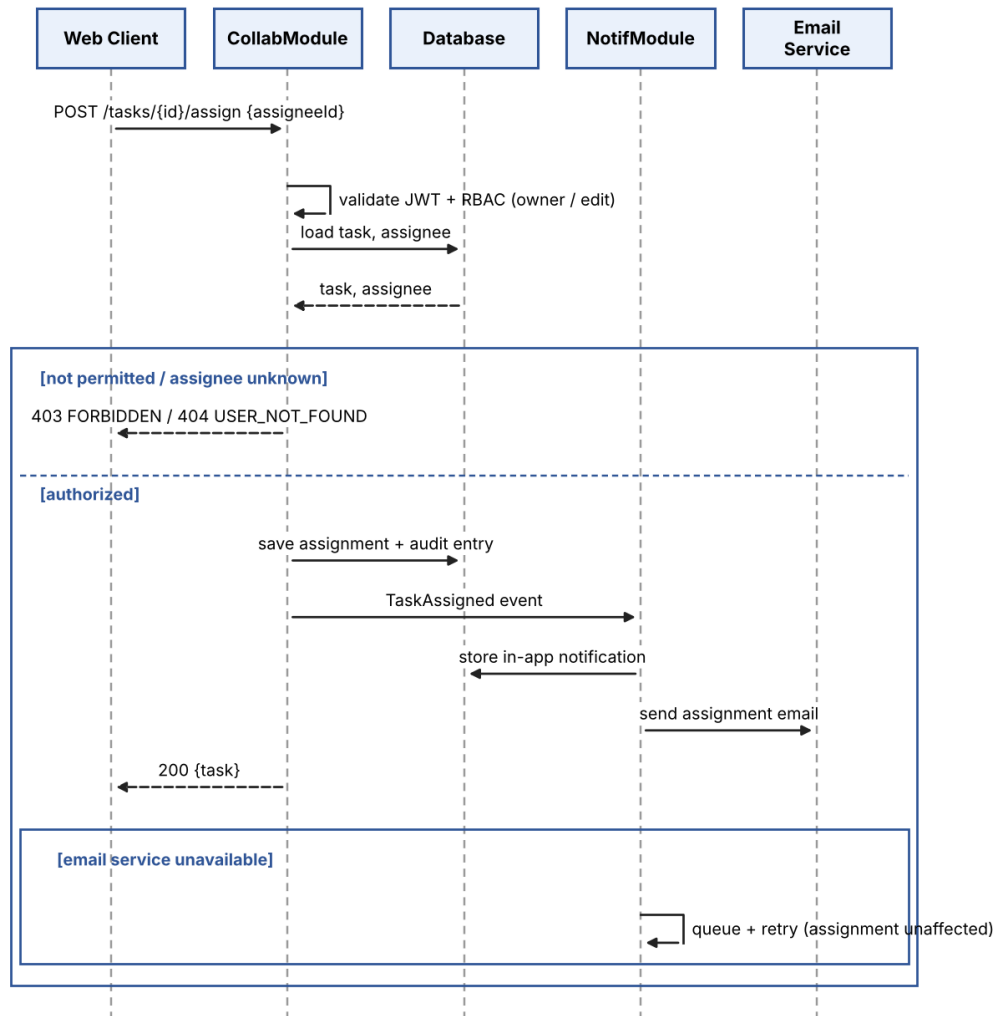


Figure 3: Sequence diagram, assigning a task and notifying the teammate (TM-F-020, TM-F-031)

4.3 API Design

Interface definition for the Auth, Task, Collaboration, Reporting and Administration components. Base path /api/v1, JSON over HTTPS. All endpoints except register, login and password reset require the header Authorization: Bearer <JWT>.

Endpoint summary

Component	Method & Endpoint	Purpose	Req
Auth	POST /auth/register	Create account	TM-F-001
Auth	POST /auth/login	Log in, returns JWT	TM-F-002/003
Auth	POST /auth/logout	Invalidate token	TM-SR-004
Auth	POST /auth/password-reset/request, /confirm	Send and use reset link	TM-F-004
Task	POST /tasks	Create task	TM-F-010
Task	GET /tasks?q=&status=&priority=&tag=&due=&sort=	Search, filter, sort	TM-F-015/016
Task	PATCH /tasks/{id}	Edit task	TM-F-011
Task	DELETE /tasks/{id}	Delete (soft delete)	TM-F-012
Task	POST /tasks/{id}/complete, /reopen	Change completion state	TM-F-013
Collab	POST /tasks/{id}/assign	Assign to a user	TM-F-020
Collab	POST /tasks/{id}/comments	Add comment	TM-F-021
Collab	POST /lists/{id}/shares	Share list (VIEW/EDIT)	TM-F-022
Report	GET /dashboard/summary	Counts by status	TM-F-040
Report	GET /tasks/export?format=csv pdf	Export report	TM-F-041
Admin	GET /admin/users; PATCH /admin/users/{id}/status; DELETE /admin/users/{id}	Manage accounts (ADMIN only)	TM-F-050
Admin	GET /admin/audit-logs	View audit log	TM-F-051

Detailed definition: AuthModule

Endpoint: /auth/login

Method: POST

Request: {email, password}

Response (200): {token, expiresIn}

Errors: 400 Validation error, 401 Invalid credentials, 423 Account locked (after 5 consecutive failed attempts)

Detailed definition: TaskModule

Endpoint: /tasks

Method: POST

Request: {title, description, dueDate, priority, tags[]}

Response (201): {id, title, description, dueDate, priority, status, tags[], createdAt}

Errors: 400 Missing title or invalid date/priority, 401 Missing or expired token

Endpoint: /tasks/{id}

Method: DELETE

Request: header Authorization; client must show a confirmation prompt first

Response (204): no body; task is soft-deleted and recoverable for 30 days

Errors: 401 Unauthorised, 403 Not the owner or no edit permission, 404 Task not found

Detailed definition: CollabModule

Endpoint: /tasks/{id}/assign

Method: POST

Request: {assigneeId}

Response (200): {task}

Errors: 401 Unauthorised, 403 Caller is not owner or editor, 404 Task or user not found

4.4 Error Handling, Logging & Monitoring

Error handling: all errors use one JSON format {status, code, message, requestId}; validation errors list the offending fields. Messages never reveal whether an email exists or any internal detail (stack traces stay in server logs). Standard codes: 400 validation, 401 unauthenticated, 403 forbidden, 404 not found, 409 conflict, 423 locked, 429 too many requests, 500 internal error.

Logging: structured logs with request ID, user ID, endpoint and outcome. Passwords, tokens and reset links are never logged. Failed logins and all admin actions are written to the audit log (TM-F-051, TM-SR-005).

Monitoring: response-time percentiles (target $p95 \leq 2$ s), error rate, uptime/health-check status (target 99.5% monthly), failed-login and lockout counts with alerts on repeated failures, notification delivery failures and retry queue length.

4.5 UX Design

Responsive layout that works on desktop and on mobile screens down to 360 px width (TM-NF-006). WCAG 2.1 AA (TM-NF-004): sufficient colour contrast, full keyboard navigation, visible focus states, labelled form fields and ARIA roles, error messages that do not rely on colour alone, and text alternatives for non-text content. Main screens: login/registration, task list with search, filter and sort controls, task detail with comments, dashboard with status counts, share/assign dialogs, notification list and admin console. Deleting a task always shows a confirmation prompt (TM-F-012).

4.6 Open Issues & Next Steps

- Confirm the technology stack with the team and course coordinator (Section 3.6 is a proposal).
- Decide the lockout duration and the reminder lead time (for example 24 hours before due date), which are not specified in the requirements.
- Choose the Notification/Email Service provider and set up its sandbox for testing.
- Finalise the PDF export layout and the database indexing plan after load tests.
- Future enhancements (outside v1.0): calendar integration, mobile push notifications, recurring tasks.

5. Appendices

5.1 Glossary: see Section 1.4.

5.2 References: Task Manager Application SRS v1.0; Software Test Plan v1.0; IEEE 42010; OWASP ASVS and Top 10; WCAG 2.1 AA; NIST SP 800-63B (authentication).

5.3 Tools: PlantUML, draw.io, Postman (API), Selenium, JMeter, Jira, axe/WAVE.